Week 2: Decision Trees and Random Forests

KM

October 11, 2026

Contents

1 Introduction **3**

2 Week 2 Theory **3**

2.1 Decision Trees . . . 3
2.2 How a Decision Tree Chooses a Split . . . 3
2.3 Regression Trees versus Classification Trees . . . 4
2.4 Gini Impurity . . . 4
2.5 Entropy . . . 4
2.6 Tree Depth . . . 4
2.7 Leaf Nodes and Minimum Samples per Leaf . . . 5
2.8 Overfitting . . . 5
2.9 Random Forest . . . 5
2.10 Bootstrap Sampling . . . 5
2.11 Random Feature Selection . . . 5
2.12 Why Random Forests can Reduce Overfitting . . . 6

3 Data Preparation **6**

4 Experimental Setup **6**

5 Decision Tree Regression **7**

5.1 Model Configuration . . . 7
5.2 Depth Experiment . . . 7
5.3 Effect of Increasing Tree Depth . . . 7
5.4 Best Decision Tree Configuration . . . 8

6 Random Forest Regression **8**

6.1 Experimental Configuration . . . 8
6.2 Random Forest Results . . . 8

7 Regression Model Comparison **9**

8 Decision Tree Classification **10**
8.1 Target and Data Leakage 10
8.2 Classification Depth Experiment 10

9 Random Forest Classification **10**

10 Classification Model Comparison **11**

11 Random Forest Model Interpretation **11**
11.1 Feature Importance 11
11.2 Classification Feature Importance 12
11.3 Feature Importance Plot 13

12 Final Questions **13**
12.1 Why can a Decision Tree overfit easily? 13
12.2 How does a Random Forest reduce overfitting? 14
12.3 Why does a Random Forest usually perform better than a single Decision Tree? 14
12.4 What is the disadvantage of using many trees? 14
12.5 Which model worked best for house-price prediction? 14
12.6 Which model worked best for high-price classification? 15
12.7 Did the most important features match what was expected? 15
12.8 When might you prefer a simple linear/logistic regression over a Random Forest? 15

13 Overall Discussion **15**

14 Limitations **16**

15 Conclusion **16**

16 Appendix: Main Week 2 Models **17**
16.1 Decision Tree Regression 17
16.2 Random Forest Regression 17
16.3 Decision Tree Classification 17
16.4 Random Forest Classification 18

17 Appendix: Software **18**

1 Introduction

This report continues the ML analysis from Week 1 using the same housing dataset. The main focus of Week 2 is on *Decision Trees* and *Random Forests* for both regression and classification. The Week 1 report already describes the dataset, data-quality problems, cleaning procedures, EDA, train/test splitting, categorical encoding, missing-value treatment and the baseline Linear and Logistic Regression models. These procedures were therefore reused rather than described again in full.

The main objectives of Week 2 are to investigate Decision Tree Regression; study the effect of tree depth and other tree parameters; identify evidence of overfitting; train and tune a Random Forest Regression model; compare Linear Regression, Decision Tree and Random Forest for house-price prediction; investigate Decision Tree Classification; train a Random Forest Classification model; compare Logistic Regression, Decision Tree and Random Forest for high-price classification; and interpret Random Forest feature importance.

2 Week 2 Theory

2.1 Decision Trees

A Decision Tree is a supervised ML model that makes predictions by repeatedly dividing the data according to feature values. Each internal node represents a decision rule, each branch represents the result of a decision, and each leaf node contains the final prediction.

For example, a tree predicting house prices might first divide houses according to living area. It could then divide one of the resulting groups according to lot size. The tree, thus, represents a sequence of decisions rather than one global mathematical equation.

2.2 How a Decision Tree Chooses a Split

At each node, the algorithm considers possible splits and chooses one that produces "child nodes" that are more homogeneous with respect to the target. The splitting criterion depends on the task.

For regression, the Decision Tree Regressor can use Mean Squared Error (MSE). For a node containing observations with target values $y_1, \ldots, y_n$, the mean squared error is

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \bar{y})^2,$$

where $\bar{y}$ is the mean target value within the node.

The algorithm chooses splits that reduce the impurity of the resulting child nodes.

2.3 Regression Trees versus Classification Trees

A `DecisionTreeRegressor` is used when the target is continuous. In this assignment, the target is $y =$ `sale_price_eur`. The prediction at a leaf is generally based on the average target value of the training observations that reach that leaf.

A `DecisionTreeClassifier`, in contrast, predicts a discrete class. In this assignment, $y =$ `high_price`, where the target represents whether a house belongs to the high-price category. Classification trees can use criteria such as *Gini impurity* or *entropy* to determine the quality of a split.

2.4 Gini Impurity

Gini impurity measures how mixed the classes are within a node.

For K classes,

$$\text{Gini impurity} = 1 - \sum_{k=1}^{K} p_k^2,$$

where p_k is the proportion of observations belonging to class k.

For a binary classification problem, a node containing observations from only one class has Gini impurity equal to zero. A node containing a mixture of classes has a higher impurity.

2.5 Entropy

Entropy is another measure of impurity:

$$\text{Entropy} = -\sum_{k=1}^{K} p_k \log_2(p_k).$$

A pure node has entropy equal to zero. Higher entropy indicates a more mixed collection of classes.

The `DecisionTreeClassifier` used in this assignment uses the default Gini criterion, unless another criterion is explicitly specified.

2.6 Tree Depth

Tree depth describes the maximum number of decision levels in the tree.

A shallow tree contains relatively few decision rules and therefore has limited complexity. A deeper tree can represent more complicated relationships.

However, increasing depth can eventually cause the model to learn very specific patterns in the training data rather than general patterns that apply to unseen houses.

2.7 Leaf Nodes and Minimum Samples per Leaf

A leaf node is a terminal node where the tree produces its final prediction.

The parameter `min_samples_leaf` controls the minimum number of training observations that must be present in a leaf. A larger minimum number of samples per leaf restricts the tree from creating very small leaves. This can reduce model complexity and may help reduce overfitting.

Similarly, `min_samples_split` controls the minimum number of observations required for a node to be considered for splitting.

2.8 Overfitting

A Decision Tree can overfit because it can continue splitting the training data until individual observations or very small groups are represented by separate leaves. A common indication of overfitting is very strong training performance combined with substantially worse test performance.

To that end, the depth experiments in this assignment compare training and test performance, instead of considering test performance alone.

2.9 Random Forest

A Random Forest is an ensemble model consisting of many Decision Trees. Instead of relying on one tree, the forest combines predictions from multiple trees.

For regression, the individual tree predictions are averaged.

For classification, the trees collectively determine the predicted class, typically through majority voting.

2.10 Bootstrap Sampling

Random Forests use bootstrap sampling to create different training samples for individual trees. Each tree therefore *sees* a slightly different sample of the training observations.

This produces diversity between the trees.

2.11 Random Feature Selection

Random Forests also introduce randomness by considering only a subset of available features when constructing individual splits.

This reduces the dependence between individual trees. If all trees were built in exactly the same way using exactly the same information, combining them would provide less benefit.

2.12 Why Random Forests can Reduce Overfitting

A single Decision Tree can be highly sensitive to the training data. Random Forests reduce this sensitivity by averaging many trees that were trained using different samples and feature subsets.

Individual trees may therefore make different errors, while averaging their predictions can produce a more stable final model.

3 Data Preparation

The same original corrupted dataset from Week 1 was used. The preprocessing procedure described in the Week 1 report was repeated from the original dataset rather than starting from a previously cleaned dataset. This is important because Week 2's assignment was required to start again from the original corrupted data.

The following procedures were therefore retained from Week 1: conversion of "?" values to missing values; correction of malformed numerical values; standardisation of inconsistent categorical values; conversion of Boolean variables to binary values; removal of duplicate property records; median imputation for numerical variables; most-frequent imputation for categorical variables; and one-hot encoding of categorical predictors.

These decisions are described in detail in the Week 1 report and are not repeated here.

The `house_id` variable was excluded because it is an identifier rather than a property characteristic.

For regression, the target remained `sale_price_eur` while that for classification remained `high_price`.

As in Week 1, `sale_price_eur` was excluded from the classification predictors because `high_price` is derived directly from sale price.

4 Experimental Setup

The same 80/20 train/test split and random state used in Week 1 were retained:

```
train_test_split(..., test_size=0.20, random_state=42)
```

For classification, the split was stratified so that the proportions of the two classes were maintained in the training and test sets.

The preprocessing steps were implemented using a `ColumnTransformer` and model pipelines. This means that imputation and encoding were fitted using the training data and then applied to the test data.

No scaling was applied to the Decision Tree or Random Forest models. Tree-based models do not require numerical variables to be placed on a common scale because their decisions are based on threshold splits.

5 Decision Tree Regression

5.1 Model Configuration

A `DecisionTreeRegressor` was used to predict `sale_price_eur`. The main parameters investigated were `max_depth`, `min_samples_split` and `min_samples_leaf`.

The purpose of the experiment was not simply to obtain one result, but to examine how changing the complexity of the tree affects generalisation.

5.2 Depth Experiment

Four Decision Tree configurations were examined. The main comparison focused on tree depth.

Table 1: *Decision Tree regression configurations*

Configuration	Max. depth	Min. split	Min. leaf	Description
Shallow tree	5	10	5	Low depth
Medium tree	10	5	2	Moderate depth
Deep tree	None	2	1	Unrestricted depth

The approximate test results were:

Table 2: *Decision Tree regression results*

Configuration	MAE (EUR)	RMSE (EUR)	R^2
Shallow tree	164,646	229,826	0.678
Medium tree	172,903	242,878	0.641
Deep tree	186,233	258,641	0.592

The shallow tree configuration produced the lowest RMSE among the tested Decision Tree configurations. The deep tree performed the worst on the test data among the three configurations, despite being more complex.

5.3 Effect of Increasing Tree Depth

The results of the depth experiment showed that the shallow tree configuration produced the lowest test RMSE among the tested Decision Tree configurations. The medium tree produced a test RMSE of approximately EUR 242,878, while the deep tree produced a test RMSE of approximately EUR 258,641.

The deep tree achieved a training RMSE of approximately EUR 0 and a training R^2 of 1.000. This indicates that it fitted the training observations almost perfectly, while its test performance was considerably worse.

Therefore, there is evidence of overfitting: **YES**. The evidence is the large difference between training and test performance for the unrestricted tree. The result suggests that a moderately constrained tree can generalise better than a tree allowed to grow without restriction.

5.4 Best Decision Tree Configuration

Among the tested configurations, the shallow tree gave the lowest test RMSE.

The result can be summarised as follows: 1) The shallow tree provided the best test performance among the tested configurations. 2) The medium tree did not improve upon the shallow tree. 3) The deep tree did not improve upon the shallow tree. 4) The unrestricted tree became too flexible and showed clear evidence of overfitting.

This demonstrates the importance of controlling tree complexity.

6 Random Forest Regression

6.1 Experimental Configuration

A `RandomForestRegressor` was trained using the same features and target as the Decision Tree Regression experiment.

The parameters investigated were `n_estimators` (number of trees), `max_depth` (maximum depth of each tree), `min_samples_leaf` (minimum observations in a leaf) and `max_features` (number or proportion of features considered when splitting).

Three configurations were compared.

Table 3: *Random Forest regression configurations*

Configuration	Trees	Max. depth	Min. leaf samples	Max. Features
RF 100 trees	100	None	1	sqrt
RF 300 trees	300	None	1	sqrt
RF tuned	300	15	2	0.7

6.2 Random Forest Results

Table 4: *Random Forest regression results*

Configuration	MAE (EUR)	RMSE (EUR)	R^2
RF 100 trees	134,890	183,364	0.795
RF 300 trees	135,303	182,791	0.796
RF tuned	133,729	182,442	0.797

The tuned configuration produced the lowest test MAE and RMSE and the highest R^2 among the tested Random Forest configurations.

The configuration was:

```
RandomForestRegressor(
n_estimators=300,
max_depth=15,
min_samples_leaf=2,
max_features=0.7,
random_state=42
)
```

It achieved approximately MAE = EUR 133, 729, RMSE = EUR 182, 442 and $R^2 = 0.797$. The result indicates that, for this experiment, the tuned configuration produced the best test performance among the tested configurations.

7 Regression Model Comparison

The Week 1 Linear Regression model provides a useful baseline. Its reported test performance was approximately MAE = EUR 100, 874, RMSE = EUR 140, 585 and $R^2 = 0.880$.

The best Decision Tree and Random Forest results from Week 2 can therefore be compared with this baseline.

Table 5: *Comparison of regression models*

Model	MAE (EUR)	RMSE (EUR)	R^2
Linear Regression	100,874	140,585	0.880
Decision Tree	164,646	229,826	0.678
Random Forest	133,729	182,442	0.797

For this particular train/test experiment, the Linear Regression model had the lowest test MAE and RMSE and the highest R^2 of the three models.

The Random Forest nevertheless improved substantially over the single Decision Tree. This is consistent with the idea that averaging many trees can produce a more stable model than relying on one tree.

The result also shows that a more flexible model does not automatically produce better test performance. The Random Forest can represent non-linear relationships, but the linear model performed better on this particular dataset and split.

8 Decision Tree Classification

8.1 Target and Data Leakage

The classification target was `high_price`.

As explained in Week 1, `sale_price_eur` was excluded from the input features because it directly determines whether a house is in the high-price category.

Therefore, X = *house characteristics* and y = `high_price`. The same restriction was applied to both the Decision Tree and Random Forest classifiers.

8.2 Classification Depth Experiment

A `DecisionTreeClassifier` was trained using several maximum-depth values.

The tested depths were 3, 5, 8 and unrestricted depth.

Table 6: *Decision Tree classification depth experiment*

Max. depth	Accuracy	Precision	Recall	F1
3	0.850	0.914	0.842	0.877
5	0.863	0.916	0.862	0.888
8	0.842	0.919	0.822	0.868
None	0.846	0.908	0.842	0.874

The depth-5 classifier achieved the highest test accuracy and F1 score among the tested configurations.

The results provide *some* evidence of overfitting for the deeper trees, as the unrestricted tree performed worse on the test data than the depth-5 configuration. The depth-5 configuration provided a better balance between fitting the training data and generalising to unseen observations.

9 Random Forest Classification

Experimental Configuration

A `RandomForestClassifier` was trained using the same features and target as the Decision Tree classifier.

The same general parameter combinations used for the regression experiment were investigated.

Table 7: *Random Forest classification results*

Configuration	Accuracy	Precision	Recall	F1
100 trees, sqrt features	0.908	0.933	0.921	0.927
300 trees, sqrt features	0.904	0.927	0.921	0.924
300 trees, tuned	0.908	0.922	0.934	0.928

The tuned configuration using 300 trees, a maximum depth of 12, a minimum leaf size of 2 and 70% of the available features produced the highest test F1 score among the tested Random Forest configurations.

10 Classification Model Comparison

The Week 1 Logistic Regression results provide the baseline for the Week 2 classification models.

The reported Week 1 results were: Accuracy = 0.929, Precision = 0.941, Recall = 0.947 and $F1 = 0.944$. The best Week 2 results can therefore be summarised as follows.

Table 8: *Comparison of classification models*

Model	Accuracy	Precision	Recall	F1
Logistic Regression	0.929	0.941	0.947	0.944
Decision Tree	0.863	0.916	0.862	0.888
Random Forest	0.908	0.922	0.934	0.928

For this experiment, Logistic Regression produced the highest values among the three models for accuracy, precision, recall and $F1$ score.

The Random Forest substantially improved over the single Decision Tree. This suggests that combining multiple trees provided more stable classification performance than relying on one tree.

The comparison also demonstrates that the more complex Random Forest does not necessarily outperform Logistic Regression on every metric.

11 Random Forest Model Interpretation

11.1 Feature Importance

Random Forest models provide a feature-importance measure that indicates how much each transformed feature contributes to reducing impurity across the forest.

The following table shows the ten most important features for the Random Forest regression model.

Table 9: *Top 10 Random Forest regression features*

Feature	Importance
Living area	0.449
Lot size	0.156
Munich	0.134
Leipzig	0.038
Bedrooms	0.027
Distance to city centre	0.026
Year built	0.019
Crime rate	0.019
School rating	0.018
Maxvorstadt	0.009

Living area was the most important feature, with an importance of approximately 0.449. Lot size was the second most important feature, followed by the Munich city indicator.

These results are broadly consistent with the Week 1 EDA, where living area, bedrooms and lot size showed relatively strong numerical relationships with sale price.

The importance values should not be interpreted as causal effects. They indicate the contribution of features to the Random Forest's prediction process rather than proving that changing a particular feature causes a specific change in house price.

11.2 Classification Feature Importance

The top features for the Random Forest classification model were also examined.

Table 10: *Top 10 Random Forest classification features*

Feature	Importance
Lot size	0.325
Living area	0.274
Munich	0.071
Crime rate	0.037
Year built	0.036
Distance to city centre	0.032
School rating	0.027
Bedrooms	0.023
Leipzig	0.019
Westend	0.018

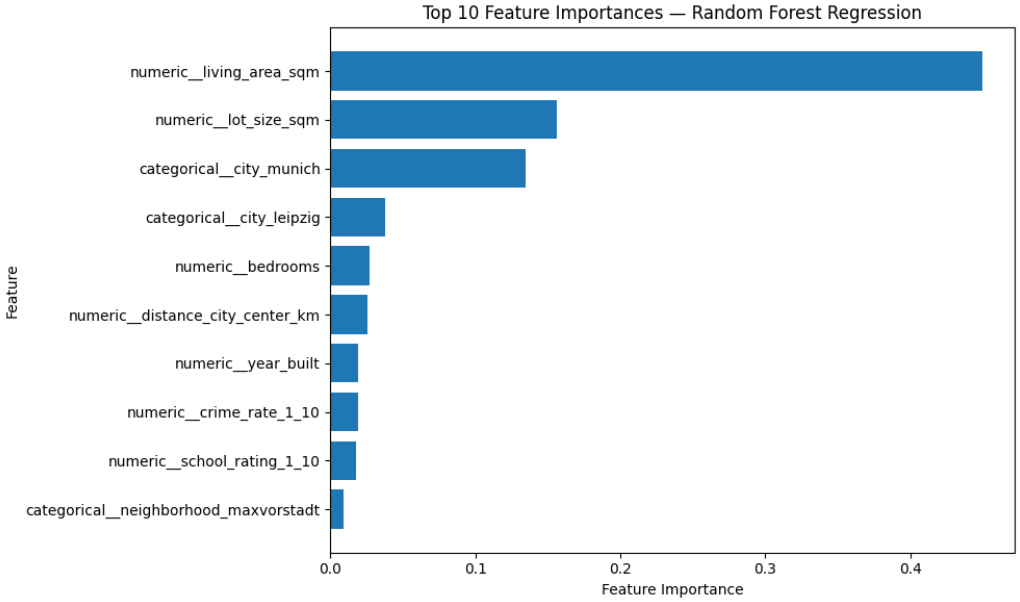
Lot size and living area were the two most important features for classifying whether a house was high-priced.

This is consistent with the regression results and with the relationships identified during the Week 1 EDA.

The appearance of city and neighbourhood indicators also suggests that location contributes useful information when distinguishing between high-priced and non-high-priced properties.

11.3 Feature Importance Plot

The Python notebook contains a bar chart of the ten most important Random Forest features.



The figure visually confirms that living area and lot size contribute substantially more to the Random Forest regression model than many of the remaining predictors.

12 Final Questions

12.1 Why can a Decision Tree overfit easily?

A Decision Tree can overfit because it can continue creating additional splits until it models very specific patterns in the training data. Deep trees may create very small leaves containing only a few observations. This can result in excellent training performance but poorer performance on unseen data.

The unrestricted regression tree in this experiment illustrates this behaviour because it achieved a training RMSE of approximately zero while its test RMSE increased to approximately EUR 258,641.

12.2 How does a Random Forest reduce overfitting?

A Random Forest reduces overfitting by combining many Decision Trees trained using different bootstrap samples and random subsets of features.

Because the individual trees are not identical, their errors are not perfectly correlated. Averaging their predictions for regression or combining their votes for classification produces a more stable model.

12.3 Why does a Random Forest usually perform better than a single Decision Tree?

A single Decision Tree can be highly sensitive to the particular training data. A Random Forest reduces this sensitivity by averaging the predictions of many different trees.

The Week 2 results provide an example of this behaviour. The best Random Forest regression model achieved an RMSE of approximately EUR 182,442, compared with approximately EUR 229,826 for the best tested single Decision Tree.

For classification, the Random Forest also achieved an F1 score of approximately 0.928 compared with approximately 0.888 for the best tested Decision Tree.

12.4 What is the disadvantage of using many trees?

The main disadvantage is increased computational cost.

A Random Forest containing hundreds of trees requires more memory and processing time than a single Decision Tree. Predictions also require the model to evaluate many trees.

After a sufficiently large number of trees, the improvement in performance may become small compared with the additional computation.

12.5 Which model worked best for house-price prediction?

For the particular test split used in this assignment, Linear Regression produced the strongest test results among the three regression models. Its reported test RMSE was approximately EUR 140,585 and its R^2 was approximately 0.880.

The Random Forest was the strongest of the two Week 2 tree-based models, with a test RMSE of approximately EUR 182,442 and R^2 of approximately 0.797.

Therefore, although Random Forest provided a substantial improvement over a single Decision Tree, it did not outperform the Week 1 Linear Regression baseline on this dataset and split.

12.6 Which model worked best for high-price classification?

Again, for the particular test split, Logistic Regression produced the highest overall classification metrics among the three models. It achieved approximately 0.929 accuracy and 0.944 $F1$ score.

The Random Forest was close, with approximately 0.908 accuracy and 0.928 F1 score. The Decision Tree performed lower, with about 0.863 accuracy and 0.888 $F1$ score.

The results therefore show that the additional flexibility of tree-based models did not automatically lead to better classification performance.

12.7 Did the most important features match what was expected?

Yes, broadly.

Living area and lot size were among the most important features for the Random Forest regression model, while lot size and living area were the two most important features for the classification model.

This agrees with the Week 1 EDA, where living area and lot size showed relatively strong relationships with sale price.

Location variables such as Munich and some neighbourhood indicators were also important, which is reasonable because house prices differ substantially between locations.

12.8 When might you prefer a simple linear/logistic regression over a Random Forest?

Linear or Logistic Regression may be preferable when the relationships are reasonably well represented by a linear model and when interpretability is important.

Linear and Logistic Regression models are also generally simpler, faster to train and easier to explain using coefficients.

Random Forest may be preferable when relationships are strongly non-linear, when interactions between variables are important, or when predictive performance benefits from an ensemble of decision trees.

In this assignment, the comparison demonstrates why the choice of model should be based on empirical evaluation rather than assuming that a more complex model will always perform better.

13 Overall Discussion

The Week 2 experiments demonstrate the effect of model complexity on predictive performance.

For Decision Tree regression, increasing the depth initially improved test performance, but an unrestricted tree overfit the training data and performed worse on the test set.

For classification, the tested tree depths produced different test metrics, with the depth-5 configuration providing the highest test accuracy and F1 score among the tested configurations.

Random Forests reduced the weaknesses of a single Decision Tree by combining many trees. In both regression and classification, the Random Forest substantially improved over the corresponding single Decision Tree.

However, the Week 2 experiments also demonstrate that model complexity does not guarantee better predictive performance. The Week 1 Linear Regression and Logistic Regression baselines remained highly competitive and achieved stronger test metrics than the corresponding tree-based models for this particular dataset and split.

14 Limitations

The results should be interpreted in relation to the particular dataset, preprocessing procedure and train/test split used in the experiment.

Only a limited number of parameter configurations were tested. A larger hyperparameter search or cross-validation could identify different configurations.

Feature importance from Random Forest should also be interpreted as a measure of predictive contribution rather than causal influence.

Finally, comparisons between models are based on a single held-out test set. Cross-validation could provide a more stable estimate of generalisation performance.

15 Conclusion

Week 2 extended the Week 1 analysis by applying Decision Trees and Random Forests to both house-price regression and high-price classification.

The Decision Tree experiments demonstrated that tree depth is an important control on model complexity. A moderate depth produced better test performance than an unrestricted tree, while the unrestricted tree showed clear evidence of overfitting.

A Random Forest improved substantially over the individual Decision Tree by combining many trees and introducing bootstrap sampling and random feature selection.

For this particular experiment, the Random Forest was the stronger of the two tree-based approaches for both regression and classification. However, the Week 1 Linear and Logistic Regression models remained competitive and achieved better overall test metrics on their respective tasks.

The feature-importance analysis also provided an interpretable view of the Random

Forest models. Living area and lot size were particularly important for house-price prediction, while location variables also provided useful predictive information.

Overall, the experiments demonstrate that model selection should be based on both predictive performance and the characteristics of the problem rather than assuming that a more complex model is necessarily better.

16 Appendix: Main Week 2 Models

16.1 Decision Tree Regression

```
DecisionTreeRegressor(
max_depth=5,
min_samples_split=10,
min_samples_leaf=5,
random_state=42
)
```

This was the best-performing tested Decision Tree regression configuration.

16.2 Random Forest Regression

```
RandomForestRegressor(
n_estimators=300,
max_depth=15,
min_samples_leaf=2,
max_features=0.7,
random_state=42
)
```

This was the best-performing tested Random Forest regression configuration.

16.3 Decision Tree Classification

```
DecisionTreeClassifier(
max_depth=5,
min_samples_leaf=2,
random_state=42
)
```

This was the best-performing tested Decision Tree classification configuration.

16.4 Random Forest Classification

```
RandomForestClassifier(
n_estimators=300,
max_depth=12,
min_samples_leaf=2,
max_features=0.7,
random_state=42
)
```

This was the best-performing tested Random Forest classification configuration.

17 Appendix: Software

The Week 2 experiments were implemented in Python using Google Colab.
The main libraries were:

- `pandas` for data manipulation;
- `numpy` for numerical operations;
- `matplotlib` and `seaborn` for visualisation;
- `scikit-learn` for preprocessing;
- `DecisionTreeRegressor` and `DecisionTreeClassifier` for Decision Trees;
- `RandomForestRegressor` and `RandomForestClassifier` for Random Forests; and
- `sklearn.metrics` for model evaluation.

The complete Python notebook contains the data preparation, experiments, predictions, evaluation metrics and feature-importance plots described in this report.